

ỦY BAN NHÂN DÂN TP . HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP TUẦN

Học phần: Seminar Chuyên đề

Đề tài: Cơ sở lập trình với GenAI

Giảng viên hướng dẫn: TS. Đỗ Như Tài

Lớp: DCT122C3

Nhóm sinh viên thực hiện:

STT	Họ và tên	MSSV
1	Nguyễn Hữu Anh Khoa	3122411098
2	Võ Thành Danh	3122411024
3	Trang Gia Huy	3122411068
4	Nguyễn Lê Quỳnh Hương	3122411078

MỤC LỤC

LỜI MỞ ĐẦU	7
CHƯƠNG I: TỪ TỰ ĐỘNG HÓA ĐẾN CHU KỲ PHÁT TRIỂN PHẦN MỀM HOÀN CHỈNH: CƠ HỘI HIỆN TẠI CHO GENAI	7
1.1. Sự thay đổi trong lĩnh vực kỹ thuật phần mềm.....	8
1.2. Sự tiến hóa của các công cụ phát triển phần mềm.....	8
1.3. Sự trỗi dậy của các Mô hình ngôn ngữ lớn (LLM).....	8
1.4. Khám phá Chu kỳ phát triển phần mềm (SDLC)	9
1.5. Bộ công cụ GenAI chủ lực	9
1.6. Đánh giá lợi ích và thách thức.....	9
1.6.1. Lợi ích	10
1.6.2. Thách thức và hạn chế	10
1.7. Kết luận	10
CHƯƠNG 2: HƯỚNG DẪN NHANH VỀ OPENAI API	11
2.1. Giới thiệu về OpenAI API	11
2.2. Giao thức HTTP RESTful và Thư viện Python.....	11
2.3. Quản lý mã khóa API (API Keys)	11
2.4. Lab 2.1: Khởi tạo và Gửi yêu cầu đầu tiên đến OpenAI API	11
2.5. Cơ chế tính chi phí và Token	12
2.6. Lab 2.2: Phân tích các tham số yêu cầu	12
2.7. Lab 2.3: Xây dựng chương trình hoàn thiện mã nguồn (Code Completion)	13
CHƯƠNG 3: SỬ DỤNG GITHUB COPILOT VỚI PYCHARM, VSCODE VÀ JUPITER NOTEBOOK	15
3.1. Mở đầu	15
3.2. Phương pháp	15
3.3. Phân tích mã nguồn.....	15
3.4. Kết luận	18

CHƯƠNG IV: CÁC PHƯƠNG PHÁP TỐT NHẤT ĐỂ HỎI ĐÁP VỚI CHATGPT	19
4.1 Giới thiệu	19
4.2 Five S Framework	19
4.2.1 Structured Prompt	19
4.2.2 Surrounding information	20
4.2.3 Single Task	21
4.2.4 Specific Instructions	21
4.2.5 Short Prompt	22
4.3. Lab 4.1 – Converting PyCharm GUI steps to CLI.....	22
4.4 Lab 4.2 – Code Debugging.....	24
4.5 Kết luận	26
CHƯƠNG V: CÁC PHƯƠNG PHÁP TỐT NHẤT ĐỂ HỎI ĐÁP VỚI API OPENAI VÀ GITHUB COPILOT	28
5.1. Mở đầu	28
5.2. Phương pháp	28
5.3. Phân tích mã nguồn.....	28
5.4. Kết luận	31
DANH MỤC TÀI LIỆU THAM KHẢO	32

DANH MỤC HÌNH ẢNH

Hình 1: Vòng đời phát triển phần mềm.....	9
Hình 2: Mã nguồn Lab 2.1	12
Hình 3: Mã nguồn Lab 2.2.....	13
Hình 4: Mã nguồn Lab 2.3	14
Hình 5: Mã nguồn Lab 3.1	16
Hình 6: Mã nguồn Lab 3.2.....	17
Hình 7: Mã nguồn Lab 3.3 (phần 1).....	17
Hình 8: Mã nguồn Lab 3.3 (phần 2).....	18
Hình 9: CLI Commands Lab 4.1	23
Hình 10: Mã nguồn Lab 5.1	30
Hình 11: Mã nguồn Lab 5.2	31

DANH MỤC BẢNG

Bảng 1: Five S Framework	19
Bảng 2: Phân tích CLI Lab 4.1 theo Five S Framework.....	24
Bảng 3: Phân tích prompt Lab 4.2	24
Bảng 4: Phân tích lỗi Lab 4.2.....	26

BẢNG PHÂN CÔNG CÔNG VIỆC

STT	Họ và tên	MSSV	Nội dung công việc
1	Nguyễn Hữu Anh Khoa	3122411098	- Nhóm trưởng - Phân chia công việc, chuẩn bị cấu trúc mã nguồn, nội dung mẫu - Soạn nội dung và thực hiện Chương V
2	Võ Thành Danh	3122411024	- Thực hiện Lab 4 - Soạn nội dung Chương IV
3	Trang Gia Huy	3122411068	- Thực hiện Lab 2 - Soạn nội dung Chương I, II và báo cáo tổng
4	Nguyễn Lê Quỳnh Hương	3122411078	- Thực hiện Lab 3 và soạn nội dung Chương III - Chuẩn bị slide báo cáo thuyết trình

LỜI MỞ ĐẦU

Trong những năm gần đây, cuộc cách mạng về Trí tuệ nhân tạo (AI), đặc biệt là Trí tuệ nhân tạo tạo sinh (Generative AI - GenAI), đã tạo nên những thay đổi mang tính bước ngoặt trong nhiều lĩnh vực của đời sống xã hội. Đối với lĩnh vực kỹ thuật phần mềm, GenAI không chỉ dừng lại ở vai trò là một công cụ hỗ trợ mà đã trở thành một người bạn đồng hành, làm thay đổi tư duy và quy trình làm việc truyền thống của các lập trình viên.

Việc ứng dụng các mô hình ngôn ngữ lớn (LLMs) như GPT của OpenAI hay các công cụ hỗ trợ mã nguồn như GitHub Copilot đã mở ra những cơ hội chưa từng có. Tuy nhiên, để làm chủ được những công nghệ này một cách hiệu quả, đảm bảo chất lượng mã nguồn và tối ưu hóa chi phí vận hành, người kỹ sư cần có một cách tiếp cận hệ thống, từ việc hiểu rõ cơ chế hoạt động của API đến việc rèn luyện kỹ năng đặt câu lệnh (Prompt Engineering).

Báo cáo này được thực hiện nhằm mục đích nghiên cứu và thực hành quá trình ứng dụng GenAI vào toàn bộ chu kỳ phát triển phần mềm (SDLC). Nội dung báo cáo sẽ đi sâu vào các khía cạnh:

- Tổng quan về cơ hội của GenAI trong việc chuyển đổi từ tự động hóa đơn thuần sang hỗ trợ toàn diện vòng đời phần mềm.
- Kỹ thuật tương tác với OpenAI API thông qua lập trình Python, quản lý tài nguyên và chi phí.
- Thực hành sử dụng GitHub Copilot trên các môi trường phát triển phổ biến (VS Code, PyCharm, Jupyter Notebook).
- Xây dựng bộ quy tắc (Best Practices) để tối ưu hóa hiệu quả khi đặt câu hỏi cho ChatGPT và các công cụ hỗ trợ.

Thông qua quá trình nghiên cứu và thực hiện các bài thực hành (Labs) dựa trên tài liệu “Supercharged Coding with GenAI” của NXB Packt, báo cáo hy vọng sẽ đưa ra cái nhìn thực tiễn và những kỹ năng cần thiết để nâng cao năng suất lập trình trong kỷ nguyên AI.

Do kiến thức về lĩnh vực này vẫn đang phát triển không ngừng, báo cáo chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ Thầy để nội dung được hoàn thiện hơn.

CHƯƠNG I: TỪ TỰ ĐỘNG HÓA ĐẾN CHU KỲ PHÁT TRIỂN PHẦN MỀM HOÀN CHỈNH: CƠ HỘI HIỆN TẠI CHO GENAI

1.1. Sự thay đổi trong lĩnh vực kỹ thuật phần mềm

Kỹ thuật phần mềm không chỉ đóng góp cho ngành công nghệ mà còn là động lực cho nhiều lĩnh vực khác như tài chính, y tế, giao thông và năng lượng. Phần mềm giúp tăng năng suất thông qua tự động hóa và tối ưu hóa quy trình, đồng thời giúp giảm chi phí vận hành.

Trong những năm gần đây, Trí tuệ nhân tạo tạo sinh (GenAI) đã tiến đến một "điểm bùng nổ", làm thay đổi căn bản cách thức làm việc của các kỹ sư. Công nghệ này cho phép từ những người mới bắt đầu đến các chuyên gia dày dặn kinh nghiệm siêu tăng cường năng suất không chỉ trong việc viết mã (coding) mà còn trong toàn bộ Chu kỳ phát triển phần mềm (SDLC).

Sự phát triển này là kết quả của sự hội tụ giữa hai xu hướng:

- Sự tiến hóa liên tục của các công cụ lập trình trong nhiều thập kỷ qua.
- Sự trỗi dậy của các Mô hình ngôn ngữ lớn (LLMs) có nguồn gốc từ mạng nơ-ron và trí tuệ nhân tạo.

1.2. Sự tiến hóa của các công cụ phát triển phần mềm

Trước khi GenAI xuất hiện, ngành phần mềm đã trải qua nhiều cuộc cách mạng công cụ:

- IDE (Môi trường phát triển tích hợp): Từ Maestro I (những năm 70) đến các công cụ hiện đại như VS Code hay PyCharm, giúp tích hợp quản lý tệp, gỡ lỗi và kiểm tra cú pháp.
- Hệ thống quản lý phiên bản (VCS): Sự ra đời của Git (2005) đã thay đổi cách cộng tác mã nguồn trên toàn cầu.
- Công cụ Build và CI/CD: Các công cụ như Jenkins, Maven giúp tự động hóa việc kiểm thử và triển khai, giảm bớt các bước thủ công.
- Phân tích mã và Tối ưu hóa: Các công cụ như SonarQube hay ReSharper sử dụng so khớp mẫu để xác định "mùi mã" (code smell) và cải thiện chất lượng phần mềm.

1.3. Sự trỗi dậy của các Mô hình ngôn ngữ lớn (LLM)

Lịch sử AI bắt đầu từ năm 1956 nhưng đã trải qua nhiều "mùa đông AI" do hạn chế về tính toán. Đến nay, Deep Learning (Học sâu) đã giải quyết được các hạn chế đó.

NLP (Xử lý ngôn ngữ tự nhiên): Đã chuyển mình nhờ các mô hình ngôn ngữ có khả năng dự đoán từ tiếp theo dựa trên dữ liệu khổng lồ.

LLM: Các mô hình như GPT (OpenAI), Llama (Meta), Gemini (Google) được huấn luyện trên hàng tỷ tham số. Khi được huấn luyện trên mã nguồn (GitHub), chúng có thể viết mã, tạo tài liệu, viết test case và tái cấu trúc mã nguồn một cách hiệu quả.

1.4. Khám phá Chu kỳ phát triển phần mềm (SDLC)

Một quan niệm sai lầm phổ biến là phát triển phần mềm chỉ là viết mã. Thực tế, việc lập trình chỉ chiếm khoảng 25-35% tổng nỗ lực của dự án. GenAI có khả năng hỗ trợ xuyên suốt các giai đoạn của SDLC:

- Thu thập yêu cầu & Phân tích: Giảm tới 60% thời gian thực hiện.
- Thiết kế hệ thống: Hỗ trợ tạo bản mẫu (prototype) nhanh chóng.
- Triển khai (Coding): Tăng tốc độ viết mã và giảm các công việc lặp lại.
- Kiểm thử (Testing): Giảm 25% thời gian tạo kế hoạch kiểm thử và unit test.
- Bảo trì: Hỗ trợ giải thích mã cũ và gỡ lỗi nhanh.



Hình 1: Vòng đời phát triển phần mềm

1.5. Bộ công cụ GenAI chủ lực

Báo cáo này tập trung vào ba công cụ chính trong hệ sinh thái hiện nay:

- ChatGPT: Chatbot dựa trên AI giúp đối thoại tự nhiên, giải quyết vấn đề thông qua kỹ thuật đặt câu lệnh (Prompt Engineering).
- OpenAI API: Cho phép lập trình viên tích hợp sức mạnh của LLM trực tiếp vào ứng dụng cá nhân, kết hợp giữa mã lệnh và trí tuệ nhân tạo.
- GitHub Copilot: Đóng vai trò như một "lập trình viên cặp" (pair programmer), dự đoán và gợi ý các khối mã tiếp theo ngay trong IDE dựa trên ngữ cảnh hiện tại.

1.6. Đánh giá lợi ích và thách thức

1.6.1. Lợi ích

Tăng năng suất: Các nghiên cứu (McKinsey, Exadel) cho thấy tốc độ hoàn thành công việc tăng từ 10% đến 50%.

Cải thiện chất lượng: Mã nguồn đồng nhất hơn, ít lỗi cú pháp cơ bản và tài liệu đi kèm đầy đủ hơn.

Tập trung vào tư duy: Giảm bớt các công việc lặp lại nhàm chán, giúp lập trình viên tập trung vào giải quyết các bài toán logic phức tạp.

1.6.2. Thách thức và hạn chế

Ảo giác (Hallucination): AI có thể tạo ra mã trông có vẻ đúng nhưng thực tế lại không hoạt động hoặc chứa lỗi logic.

Sự phụ thuộc: Nguy cơ làm giảm tư duy phản biện và khả năng ghi nhớ cú pháp (cognitive offloading).

An ninh mạng: Nguy cơ lộ lọt dữ liệu bí mật khi đưa mã nguồn lên các mô hình AI công cộng.

1.7. Kết luận

GenAI không phải là công cụ để tin tưởng mù quáng, nhưng cũng không nên bị bài trừ. Kỹ năng quan trọng nhất của lập trình viên hiện đại là biết cách kiểm soát đầu vào và đánh giá đầu ra của AI để tối ưu hóa quy trình SDLC.

CHƯƠNG 2: HƯỚNG DẪN NHANH VỀ OPENAI API

Chương này tập trung vào việc thiết lập và sử dụng Giao diện lập trình ứng dụng (API) của OpenAI để tạo mã nguồn chất lượng cao. Chúng ta sẽ đi từ việc thiết lập tài khoản, hiểu về cơ chế chi phí (Token) cho đến việc xây dựng một chương trình hoàn thiện mã nguồn cơ bản.

2.1. Giới thiệu về OpenAI API

Nền tảng OpenAI cung cấp các dịch vụ xử lý ngôn ngữ tự nhiên (NLP) dựa trên các Mô hình ngôn ngữ lớn (LLM). Các nhà phát triển tương tác với nền tảng thông qua các yêu cầu dạng văn bản được gọi là Prompt.

Dịch vụ Chat của OpenAI sử dụng ba vai trò (roles) chính để điều phối hội thoại:

System (Hệ thống): Định nghĩa hành vi và quy tắc cho AI (Ví dụ: "Bạn là một giảng viên lập trình tận tâm").

User (Người dùng): Các câu hỏi hoặc yêu cầu từ phía người dùng.

Assistant (Trợ lý): Các câu trả lời được tạo ra bởi mô hình AI.

2.2. Giao thức HTTP RESTful và Thư viện Python

Mọi dịch vụ của OpenAI đều có thể truy cập qua giao thức HTTP RESTful. Tuy nhiên, đối với lập trình viên Python, việc sử dụng thư viện openai sẽ đơn giản hóa quá trình xác thực và quản lý lỗi.

2.3. Quản lý mã khóa API (API Keys)

Để sử dụng dịch vụ, lập trình viên cần khởi tạo Project API Key từ trang quản trị của OpenAI.

Lưu ý bảo mật: Không bao giờ viết trực tiếp mã khóa (hardcode) vào tệp tin .py. Thay vào đó, hãy sử dụng biến môi trường hoặc tệp .env.

2.4. Lab 2.1: Khởi tạo và Gửi yêu cầu đầu tiên đến OpenAI API

Mục tiêu của bài thực hành này là giúp sinh viên làm quen với thư viện openai trong Python và hiểu cấu trúc cơ bản của một yêu cầu Chat Completion.

Các bước thực hiện chi tiết:

- Khởi tạo Client: Sử dụng `client = OpenAI()`. Lưu ý rằng thư viện sẽ mặc định tìm kiếm biến môi trường `OPENAI_API_KEY`.
- Cấu hình tham số model: * Sử dụng `gpt-4o-mini` cho các tác vụ thông thường để tiết kiệm chi phí.
- Sử dụng `gpt-3.5-turbo` nếu tài khoản đang ở bậc dùng thử hạn chế.
- Cấu hình tham số messages: Đây là một danh sách các đối tượng (list of dictionaries). Ở mức độ cơ bản, chúng ta chỉ truyền vào một tin nhắn của người dùng (`role: "user"`).

Phân tích mã nguồn:

```
lab21.py X
lab21.py
1  import openai
2  import os
3  from openai import OpenAI
4  from dotenv import load_dotenv
5
6  if __name__ == "__main__":
7
8      load_dotenv()
9      client: OpenAI = OpenAI(
10         api_key=os.getenv("OPENAI_API_KEY")
11     )
12
13     completion: openai.ChatCompletion = client.chat.completions.create(
14         model="gpt-4o-mini",
15         messages=[{"role": "user", "content": "What is the FizzBuzz problem?"}]
16     )
17
18     print(completion.choices[0].message.content)
19
20     print("\nToken usage:")
21     print("Prompt tokens:", completion.usage.prompt_tokens)
22     print("Completion tokens:", completion.usage.completion_tokens)
23     print("Total tokens:", completion.usage.total_tokens)
```

Hình 2: Mã nguồn Lab 2.1

2.5. Cơ chế tính chi phí và Token

OpenAI không tính phí theo số lượng câu hỏi mà tính theo Token.

Token là gì? Là các đoạn nhỏ của từ (trung bình 1.000 token tương đương khoảng 750 từ).

Chi phí: Được tính trên cả đầu vào (Input) và đầu ra (Output). Các mô hình dòng "-mini" hoặc "-turbo" thường có chi phí rẻ hơn nhiều so với dòng gốc (ví dụ: gpt-4o-mini rẻ hơn khoảng 16 lần so với gpt-4o).

2.6. Lab 2.2: Phân tích các tham số yêu cầu

Bài Lab này tập trung vào việc điều khiển "hành vi" và "độ dài" của AI. Đây là bước quan trọng để cá nhân hóa kết quả cho các ứng dụng thực tế.

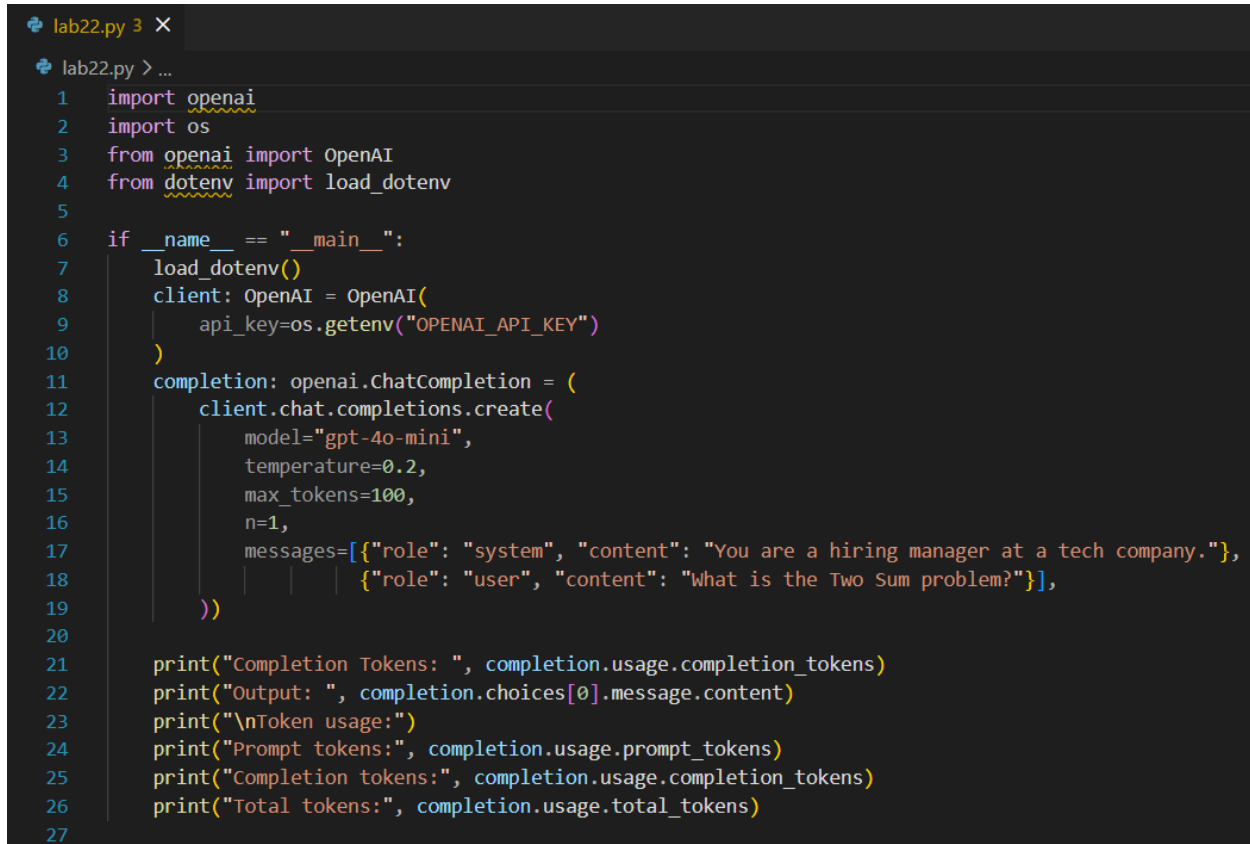
Các tham số kỹ thuật cần chú ý:

System Message: Thiết lập ngữ cảnh. Ví dụ: "Bạn là một chuyên gia tuyển dụng tại công ty công nghệ". Điều này thay đổi giọng văn của AI từ chung chung sang chuyên nghiệp và tập trung vào kỹ thuật.

Temperature (Độ nhiễu): Temperature = 0.2: Phù hợp cho lập trình và giải toán vì tính logic cao, ít sai số.

Temperature > 1.0: Phù hợp cho viết văn tạo hình, quảng cáo.

Max_tokens: Giới hạn phản hồi (ví dụ: 100 tokens). Điều này cực kỳ quan trọng để kiểm soát ngân sách khi chạy các dự án lớn.



```
lab22.py 3 X
lab22.py > ...
1  import openai
2  import os
3  from openai import OpenAI
4  from dotenv import load_dotenv
5
6  if __name__ == "__main__":
7      load_dotenv()
8      client: OpenAI = OpenAI(
9          api_key=os.getenv("OPENAI_API_KEY")
10     )
11     completion: openai.ChatCompletion = (
12         client.chat.completions.create(
13             model="gpt-4o-mini",
14             temperature=0.2,
15             max_tokens=100,
16             n=1,
17             messages=[{"role": "system", "content": "You are a hiring manager at a tech company."},
18                       {"role": "user", "content": "what is the Two Sum problem?"}],
19         )
20
21     print("Completion Tokens: ", completion.usage.completion_tokens)
22     print("Output: ", completion.choices[0].message.content)
23     print("\nToken usage:")
24     print("Prompt tokens:", completion.usage.prompt_tokens)
25     print("Completion tokens:", completion.usage.completion_tokens)
26     print("Total tokens:", completion.usage.total_tokens)
27
```

Hình 3: Mã nguồn Lab 2.2

2.7. Lab 2.3: Xây dựng chương trình hoàn thiện mã nguồn (Code Completion)

Trong Lab này, chúng ta xây dựng một công cụ nhận vào "chữ ký hàm" (Function Signature) và yêu cầu AI viết tiếp phần thân hàm.

Quy trình thực hiện:

- Định nghĩa SYSTEM_PROMPT để AI đóng vai trò lập trình viên chuyên nghiệp.
- Gói yêu cầu của người dùng kèm theo chỉ dẫn "Complete this code".
- Trích xuất mã nguồn từ khối Markdown (python ...) trong phản hồi của AI.

```

lab23.py 3 X
lab23.py > ...
1  import re
2  import os
3  import openai
4  from openai import OpenAI
5  from dotenv import load_dotenv
6
7  USER_PROMPT = """
8  def print_fibonacci_sequence(n: int) -> None:
9  """
10 SYSTEM_PROMPT = ("You will be provided with a Python function signature. Your task is to implement the function. Return "
11 | | | | | "code only.")
12
13
14 def get_code_with_instructions(code: str) -> str:
15     """
16     Add a comment to the code for specific code completion instruction
17     :param code: Python code as string
18     :return: The code with additional instruction - "Complete this code"
19     """
20
21     return code + "\n# Complete this code"
22
23
24 if __name__ == "__main__":
25     load_dotenv()
26     client: OpenAI = OpenAI(
27         api_key=os.getenv("OPENAI_API_KEY")
28     )
29
30     completion: openai.ChatCompletion = (
31         client.chat.completions.create(
32             model="gpt-4o-mini",
33             temperature=1,
34             n=2,
35             messages=[{"role": "system", "content": SYSTEM_PROMPT},
36 | | | | | {"role": "system", "content": "Include docstring and typehints."},
37 | | | | | {"role": "user", "content": get_code_with_instructions(USER_PROMPT)}],
38         ))
39
40     for i in range(2):
41         output = completion.choices[i].message.content
42         code_suggestion = re.sub(r"(.*)```python(.*)```(.*)", r"\2", output, flags=re.DOTALL).strip()
43         print(f"Output {i + 1}:")
44         print(code_suggestion)

```

Hình 4: Mã nguồn Lab 2.3

Kết luận: Việc làm chủ OpenAI API không chỉ dừng lại ở việc gửi câu hỏi, mà còn nằm ở khả năng tối ưu hóa tham số và quản lý chi phí hiệu quả. Đây là nền tảng để xây dựng các ứng dụng AI phức tạp hơn trong các chương sau.

CHƯƠNG 3: SỬ DỤNG GITHUB COPILOT VỚI PYCHARM, VSCODE VÀ JUPITER NOTEBOOK

3.1. Mở đầu

Chương 3 của cuốn sách mở ra một hành trình thực tiễn khi đưa người đọc đến với GitHub Copilot – công cụ được mệnh danh là "lập trình viên cặp AI" (AI pair programmer) tiên phong và phổ biến nhất hiện nay. Thay vì chỉ dừng lại ở các khái niệm lý thuyết về Generative AI, chương này tập trung vào việc biến AI thành một cộng sự đắc lực trong quy trình làm việc hàng ngày của nhà phát triển. Điểm khác biệt cốt lõi của Copilot so với các công cụ gợi ý mã truyền thống chính là khả năng tận dụng các mô hình ngôn ngữ lớn được huấn luyện trên khối lượng dữ liệu khổng lồ từ các kho mã nguồn mở. Mục tiêu chính của chương là trang bị cho người dùng những kỹ năng cần thiết để thiết lập, cấu hình và làm chủ các phương thức tương tác đa dạng trên những môi trường lập trình hàng đầu như PyCharm, VS Code hay Jupyter Notebook, từ đó tối ưu hóa hiệu suất và giảm bớt gánh nặng từ các công việc mang tính lặp lại.

3.2. Phương pháp

Đề khai thác tối đa sức mạnh của GitHub Copilot, chương này hướng dẫn tiếp cận qua các phương pháp hệ thống sau:

Thiết lập và Kích hoạt hệ sinh thái: Hướng dẫn quy trình đăng ký tài khoản GitHub, xác thực quyền truy cập và cài đặt các tiện ích mở rộng (extensions) tương ứng trên từng IDE (PyCharm và VS Code).

Cơ chế thu thập ngữ cảnh thông minh (Context Window): Giải thích cách Copilot xây dựng câu lệnh (prompt) dựa trên các tệp tin đang mở, cấu trúc thư mục, lịch sử Git và các chỉnh sửa gần đây để đưa ra gợi ý phù hợp nhất với dự án hiện tại.

Làm chủ ba chế độ tương tác chính:

- **Copilot Chat:** Giao diện hội thoại để đặt câu hỏi trực tiếp hoặc yêu cầu viết mã từ ngôn ngữ tự nhiên.
- **Inline Completions (Ghost Text):** Khả năng dự đoán và hiển thị mã nguồn mờ ngay tại vị trí con trỏ khi đang soạn thảo.
- **Copilot Commands:** Sử dụng các lệnh đặc biệt như `/fix` để sửa lỗi, `/explain` để giải thích mã và `/tests` để tạo kiểm thử tự động.

Kỹ năng điều khiển gợi ý: Cách sử dụng phím Tab để chấp nhận code, Ctrl + Right Arrow để nhận từng từ, và phím Alt +] / [để chuyển đổi giữa các phương án mã nguồn khác nhau mà AI đề xuất.

Tính năng Copilot Edits: Tìm hiểu khả năng chỉnh sửa đồng thời trên nhiều tệp tin thuộc cùng một bộ ngữ cảnh (Working Set), giúp duy trì tính đồng bộ cho dự án.

3.3. Phân tích mã nguồn

Nội dung thực hành trong chương tập trung vào bài toán tính toán số học để minh họa tư duy của AI:

- Xây dựng mã nguồn từ ý định (Lab 3.1): Khi người dùng khai báo hàm `get_geometric_mean_of_two_numbers`, Copilot ngay lập tức nhận diện mục tiêu và gợi ý công thức `return pow(a * b, 1/2)`, thể hiện sự nhạy bén trong việc kết nối tên hàm với logic toán học.
- Xử lý cấu trúc dữ liệu phức tạp (Lab 3.2): Với các tham số mảng (`*nums`), AI đề xuất các giải pháp sử dụng vòng lặp hoặc thư viện `math`. Qua đó, người dùng học được cách duyệt qua các gợi ý để chọn lọc giải pháp tối ưu và ngắn gọn nhất.
- Khả năng gỡ lỗi và suy luận logic (Lab 3.3): Trong môi trường Jupyter Notebook, khi đối mặt với một hàm tính toán sai logic (như tính căn bậc hai thay vì căn bậc ba cho ba số), lệnh `/fix` đã chứng minh sức mạnh khi đề xuất sửa đổi đúng về mặt bản chất toán học thay vì chỉ sửa lỗi cú pháp.
- Diễn giải mã nguồn: Thông qua lệnh `/explain`, các đoạn mã phức tạp được AI bóc tách thành các bước thực hiện rõ ràng, hỗ trợ đắc lực cho việc tiếp nhận mã nguồn cũ hoặc đào tạo nhân sự mới.

```
D: > Downloads > lab31.py > get_geometric_mean_of_two_numbers
1  def get_geometric_mean_of_two_numbers(
2      a: float,
3      b: float,
4  ) -> float:
5      return pow(a * b, 1 / 2)
6
7  num1: float = 5.0
8  num2: float = 20.0
9  print(get_geometric_mean_of_two_numbers(num1, num2))
10
```

Hình 5: Mã nguồn Lab 3.1

```

1  # your prompt here
2  def get_geometric_mean(*nums):
3      if not nums:
4          raise ValueError("At least one number is required")
5      for num in nums:
6          if not isinstance(num, (int, float)):
7              raise TypeError("All numbers must be integers or floats")
8      product = 1
9      for num in nums:
10         product *= num
11     return pow(product, 1 / len(nums))
12

```

Hình 6: Mã nguồn Lab 3.2

Generate + Code + Markdown Run All Clear All Outputs Outline ...

Lab 3.3: Code analysis with GitHub Copilot

```

def get_geometric_mean_for_three_numbers(a, b, c):
    return (a*b*c)**(1/2)

```

```

from functools import reduce

def get_geometric_mean(*nums: float) -> float:
    """
    Get the geometric mean of a sequence of numbers
    """

    if not len(nums):
        raise ValueError("Cannot calculate the geometric mean of an empty sequence")

    product = reduce(lambda a, b: a * b, nums)
    if product < 0 and len(nums) % 2 == 0:
        raise ValueError("Cannot calculate the geometric mean")

    return pow(product, 1 / len(nums))

```

Hình 7: Mã nguồn Lab 3.3 (phần 1)

```

1  from functools import reduce
2
3
4  def get_geometric_mean_for_three_numbers(a, b, c):
5      return (a*b*c)**(1/2)
6
7
8  def get_geometric_mean(*nums: float) -> float:
9      """
10         Get the geometric mean of a sequence of numbers
11         """
12
13     if not len(nums):
14         raise ValueError("Cannot calculate the geometric mean of an empty sequence")
15
16     product = reduce(lambda a, b: a * b, nums)
17     if product < 0 and len(nums) % 2 == 0:
18         raise ValueError("Cannot calculate the geometric mean")
19
20     return pow(product, 1 / len(nums))

```

Hình 8: Mã nguồn Lab 3.3 (phần 2)

3.4. Kết luận

Tóm lại, Chương 3 đã khẳng định rằng GitHub Copilot không chỉ là một công cụ hỗ trợ gõ mã nhanh mà thực sự là một cộng tác viên thông minh có khả năng hiểu sâu sắc ngữ cảnh của dự án. Việc thành thạo các chế độ tương tác và hệ thống phím tắt không chỉ giúp nhà phát triển gia tăng năng suất đáng kể mà còn biến quá trình lập trình trở nên thú vị và trôi chảy hơn. Tuy nhiên, cuốn sách cũng nhấn mạnh một thông điệp quan trọng: dù AI có mạnh mẽ đến đâu, nó vẫn chỉ đóng vai trò hỗ trợ. Lập trình viên vẫn cần giữ vững vai trò kiểm duyệt, có tư duy phản biện để đánh giá tính chính xác và an toàn của mã nguồn do AI tạo ra. Những kiến thức thực tế từ chương này chính là tiền đề vững chắc để người đọc tiếp tục khám phá nghệ thuật viết câu lệnh (Prompt Engineering) chuyên sâu hơn ở các chương kế tiếp.

CHƯƠNG IV: CÁC PHƯƠNG PHÁP TỐT NHẤT ĐỂ HỎI ĐÁP VỚI CHATGPT

4.1 Giới thiệu

Trong lập trình với Generative AI, prompt engineering là kỹ thuật viết câu lệnh để hướng dẫn mô hình AI tạo ra kết quả chính xác và hữu ích.

ChatGPT có thể hỗ trợ nhiều nhiệm vụ lập trình như:

- sinh mã nguồn
- giải thích code
- sửa lỗi (debug)
- tạo tài liệu
- viết test

Tuy nhiên, chất lượng kết quả phụ thuộc rất nhiều vào cách viết prompt.

Trong chương này, sách giới thiệu Five S Framework để tạo prompt hiệu quả.

4.2 Five S Framework

Five S Framework là tập hợp 5 nguyên tắc giúp cải thiện prompt.

Principle	Ý nghĩa
Structured	Prompt có cấu trúc rõ ràng
Surrounding context	Cung cấp ngữ cảnh
Single task	Chỉ yêu cầu một nhiệm vụ
Specific instructions	Hướng dẫn cụ thể
Short prompt	Prompt ngắn gọn

Bảng 1: Five S Framework

4.2.1 Structured Prompt

Prompt nên được chia thành các phần rõ ràng.

Ví dụ:

CONTEXT: You will be provided with a COBOL code enclosed with {{{ COBOL CODE }}}.

TASK: Your task is to convert it to Python, with the following guidelines:

...

COBOL CODE: {{{
}}}

Python Code:

Cách viết này giúp mô hình hiểu:

- đâu là input
- đâu là instruction

4.2.2 Surrounding information

Khi xác định ngữ cảnh xung quanh, hãy bao gồm thông tin về dữ liệu liên quan đến vấn đề.

Ví dụ 1 – Code explanation hoặc code generation

You are provided with Python code / a function / a function signature.

Context này có thể dùng cho nhiều nhiệm vụ như:

- giải thích code
- sửa code
- tạo docstring
- tối ưu code

Ví dụ 2 – Code debugging

You are provided with a Python function that contains a bug.

Prompt này thường dùng trong các nhiệm vụ:

- tìm lỗi chương trình
- sửa lỗi
- cải thiện logic code

Ví dụ sử dụng trong lab debugging.

Ví dụ 3 – GUI to CLI conversion

You are provided with GUI steps.

Context này dùng cho các bài toán như:

- chuyển thao tác GUI thành CLI
- mô tả workflow
- tự động hóa thao tác

4.2.3 Single Task

Luôn hướng tới việc chỉ thực hiện một nhiệm vụ duy nhất trong mỗi prompt. Cách tiếp cận này không chỉ giúp bạn dễ dàng đánh giá kết quả đầu ra của mô hình bằng cách tập trung vào một nhiệm vụ cụ thể, mà còn làm tăng khả năng mô hình đã từng gặp những nhiệm vụ tương tự trong quá trình huấn luyện, từ đó nâng cao mức độ hiểu biết và khả năng xử lý của mô hình đối với chủ đề đó.

Ví dụ, trong các nhiệm vụ tái cấu trúc mã nguồn (refactoring), không nên kết hợp nhiều mục tiêu trong cùng một prompt, như ví dụ sau:

Explain the following function and fix compilation bugs.

Mặc dù các mô hình ngôn ngữ lớn (LLMs) được huấn luyện tốt cho từng nhiệm vụ riêng lẻ, nhưng việc kết hợp nhiều nhiệm vụ trong một prompt có thể ít phổ biến hơn trong dữ liệu huấn luyện. Điều này có thể làm giảm khả năng mô hình xử lý tốt yêu cầu.

Thay vào đó, nên tập trung vào một nhiệm vụ duy nhất, chẳng hạn như:

- giải thích hàm (explain the function), hoặc
- sửa lỗi biên dịch (fix compilation bugs).

Cách tiếp cận này giúp mô hình đưa ra kết quả chính xác và rõ ràng hơn.

4.2.4 Specific Instructions

Là các lập trình viên phần mềm, chúng ta thường có xu hướng sử dụng các từ khóa như optimize (tối ưu hóa), improve (cải thiện), refactor (tái cấu trúc), hoặc scale (mở rộng). Tuy nhiên, những từ này thường quá rộng và mơ hồ, khiến mô hình có thể hiểu theo nhiều cách khác nhau.

Để đạt được kết quả tốt hơn, nên sử dụng các hướng dẫn cụ thể, chẳng hạn như:

- Sử dụng list comprehension thay cho for loop.
- Sử dụng np.array để vector hóa các phép tính trong vòng lặp for.
- Chuyển các giá trị số được hard-code thành hằng số toàn cục (global constants).
- Lưu cache kết quả của hàm bằng lru_cache.

Các hướng dẫn cụ thể giúp loại bỏ sự mơ hồ, đảm bảo rằng mô hình hiểu chính xác những thay đổi hoặc hành động cần thực hiện. Nhờ đó, khả năng mô hình tạo ra kết quả đúng với mong muốn sẽ cao hơn.

4.2.5 Short Prompt

Prompt nên ngắn gọn nhưng đầy đủ thông tin.

Prompt quá dài hoặc lan man sẽ làm mô hình khó hiểu yêu cầu.

4.3. Lab 4.1 – Converting PyCharm GUI steps to CLI

Mục tiêu

- Sử dụng ChatGPT để chuyển các thao tác GUI trong PyCharm sang lệnh CLI của Git.

Prompt sử dụng

CONTEXT: You will be provided with PyCharm GUI steps enclosed with {{{ STEPS }}} to execute a process enclosed with {{{ PROCESS }}}.

TASK: Convert the steps to CLI commands.

PROCESS: {{{ Commit and push files to a remote Git branch }}}}

STEPS: {{{

1. Review changed files
2. Stage the desired files
3. Add a commit message
4. Commit the files
5. Validate branch name
6. Push the changes to the remote branch

}}}

CLI COMMANDS:

```
Reviewed steps for committing and pushing files to Git

Here are the CLI commands for committing and pushing files to a remote Git
branch:

# 1. Review changed files
git status

# 2. Stage the desired files
git add <file1> <file2>
# Or to stage all changed files:
git add .

# 3 & 4. Add a commit message and commit the files
git commit -m "Your commit message here"

# 5. Validate branch name
git branch --show-current

# 6. Push the changes to the remote branch
git push origin <branch-name>
# Or if tracking is already set up:
git push

Complete workflow example:

git status
git add .
git commit -m "Your commit message"
git branch --show-current
git push origin main
```

Hình 9: CLI Commands Lab 4.1

Phân tích

Prompt này áp dụng đầy đủ Five S Framework:

Principle	Áp dụng
Structured	CONTEXT, TASK, PROCESS
Surrounding context	mô tả quy trình commit

Single task	convert GUI → CLI
Specific instructions	liệt kê steps
Short prompt	prompt ngắn gọn

Bảng 2: Phân tích CLI Lab 4.1 theo Five S Framework

4.4 Lab 4.2 – Code Debugging

Mục tiêu

Sử dụng ChatGPT để tìm và sửa lỗi Python code.

Prompt sử dụng

CONTEXT: You will be provided with Python code in the Python editor with explanation enclosed with `{{{ CODE_EXPLANATION }}}}` that may contain errors enclosed with `{{{ ERRORS}}}`.

TASK: Fix the errors in the code.

CODE_EXPLANATION: `{{{`

Run 10 random multiplication quizzes and provide feedback to the user

`}}}`

ERRORS: `{{{`

1. Compilation

2. Reproducibility of random number generation

3. Logical scoping of variable assignments

4. Error handling for input casting

`}}}`

Phân tích prompt

Principle	Áp dụng
Structured	prompt có instruction rõ
Surrounding context	có code input
Single task	chỉ yêu cầu 1 thay đổi
Specific instructions	mô tả chính xác
Short prompt	prompt ngắn

Bảng 3: Phân tích prompt Lab 4.2

Code sau khi sửa

```
import random

def run_multiplication_quiz():
    """Run 10 random multiplication quizzes and provide feedback about each to
    the user."""

    # Fix #2: Set seed for reproducibility of random number generation
    random.seed(42)

    # Fix #3: Proper scoping of variable assignments
    correct_count = 0
    total_questions = 10

    print("Welcome to the Multiplication Quiz!")
    print("You will be asked 10 multiplication questions.\n")

    for i in range(1, total_questions + 1):
        # Generate two random numbers between 1 and 12
        num1 = random.randint(1, 12)
        num2 = random.randint(1, 12)
        correct_answer = num1 * num2

        print(f"Question {i}: What is {num1} × {num2}?")

        # Fix #4: Error handling for input casting
        try:
            user_answer = int(input("Your answer: "))

            if user_answer == correct_answer:
                print("✓ Correct!\n")
                correct_count += 1
            else:
```

```

        print(f"X Incorrect. The correct answer is {correct_answer}.\n")
    except ValueError:
        print(f"X Invalid input. The correct answer was {correct_answer}.\n")

    # Provide final feedback
    percentage = (correct_count / total_questions) * 100
    print(f"Quiz completed! You got {correct_count} out of {total_questions} correct ({percentage:.1f}%).")

    if percentage >= 90:
        print("Excellent work!")
    elif percentage >= 70:
        print("Good job!")
    elif percentage >= 50:
        print("Not bad, but keep practicing!")
    else:
        print("Keep practicing to improve your skills!")

if __name__ == "__main__":
    # Fix #1: No compilation errors - proper syntax throughout
    run_multiplication_quiz()

```

Phân tích lỗi

Lỗi	Cách sửa
Compilation error	sửa syntax
Random reproducibility	thêm random.seed()
Variable scoping	đặt biến ngoài loop
Input casting	dùng try/except

Bảng 4: Phân tích lỗi Lab 4.2

4.5 Kết luận

Prompt engineering đóng vai trò quan trọng khi sử dụng ChatGPT cho lập trình.

Five S Framework giúp cải thiện chất lượng kết quả bằng cách cung cấp prompt rõ ràng, có cấu trúc và đầy đủ ngữ cảnh.

Thông qua hai bài lab minh họa:

- Lab 4.1 cho thấy ChatGPT có thể chuyển đổi các thao tác GUI sang CLI.
- Lab 4.2 cho thấy ChatGPT có thể hỗ trợ phát hiện và sửa lỗi trong chương trình Python.

Việc áp dụng các nguyên tắc prompting giúp tăng độ chính xác và hiệu quả khi sử dụng GenAI trong phát triển phần mềm

CHƯƠNG V: CÁC PHƯƠNG PHÁP TỐT NHẤT ĐỂ HỎI ĐÁP VỚI API OPENAI VÀ GITHUB COPILOT

5.1. Mở đầu

Chương 5 là mảnh ghép hoàn thiện cho phần nền tảng về kỹ thuật nhắc lệnh (Prompting) bằng cách mở rộng các phương pháp từ ChatGPT sang hai công cụ chuyên sâu hơn là OpenAI API và GitHub Copilot. Nếu như các chương trước tập trung vào việc làm quen với giao diện, thì chương này đi sâu vào kỹ thuật tinh chỉnh để tạo ra kết quả có tính dự báo cao và chất lượng vượt trội. Mục tiêu trọng tâm của chương là giúp nhà phát triển làm chủ khung quy tắc "5 chữ S" (Structured, Surrounding, Single task, Specific, Short) để điều khiển AI xử lý các cấu trúc mã nguồn phức tạp như Class, Decorator hay các mẫu thiết kế (Design Patterns). Bằng cách hiểu rõ sự khác biệt trong cách tiếp nhận ngữ cảnh của API so với Plugin trong IDE, người đọc sẽ học được cách tối ưu hóa chi phí và hiệu suất khi tích hợp AI vào quy trình phát triển chuyên nghiệp.

5.2. Phương pháp

Để đạt được kết quả tối ưu, chương này hướng dẫn các phương pháp thiết lập câu lệnh một cách hệ thống cho từng công cụ:

Đối với OpenAI API:

- Cấu trúc hóa (Structured): Sử dụng các thuộc tính trích xuất từ đối tượng Python để làm đầu vào cho API.
- Phân loại vai trò (Role-based): Phân định rõ ràng giữa "System prompt" (thiết lập bối cảnh chung, nhiệm vụ duy nhất) và "User prompt" (chứa các chỉ dẫn cụ thể cho tác vụ hiện tại).

Đối với GitHub Copilot:

- Sử dụng gợi ý dẫn dắt (Lead-in cue): Bắt đầu bằng một đoạn mã mời (ví dụ: tên class hoặc hàm) để định hướng cho AI.
- Tận dụng bối cảnh xung quanh (Surrounding context): Khai thác các dòng lệnh import, thẻ hashtag hoặc các tệp tin đang mở để thu hẹp phạm vi suy luận của Copilot.
- Áp dụng gợi ý kiểu (Type hints): Sử dụng các chú thích kiểu dữ liệu và docstring để AI hiểu rõ cấu trúc dữ liệu đầu vào và đầu ra.
- Nguyên tắc ngắn gọn (Short prompts): Loại bỏ các bình luận thừa thãi (fluff) để tránh làm nhiễu mô hình.

5.3. Phân tích mã nguồn

Các kỹ thuật trên được minh họa cụ thể thông qua hai bài thực hành chuyên sâu (Labs):

- Tự động tạo tài liệu mã nguồn (Lab 5.1):
- Sử dụng OpenAI API để tạo docstring cho một Singleton design pattern.

- AI được yêu cầu phân tích phương thức `__call__` của Singleton để viết mô tả chi tiết về cách kiểm soát quá trình khởi tạo instance, đảm bảo tài liệu phản ánh chính xác logic kỹ thuật của mẫu thiết kế này.
- Sửa lỗi thực thi với GitHub Copilot (Lab 5.2):
- Người dùng cố tình đưa ra một triển khai Singleton bị lỗi (faulty implementation).
- Thông qua chuỗi 5 câu lệnh áp dụng quy tắc 5 chữ S, AI đã nhận diện được lỗi sử dụng `cls.__name__` sai cách và đề xuất thay thế bằng `cls._instances[cls]` để quản lý instance chính xác theo kiểu dữ liệu class.
- Kết quả được kiểm chứng bằng việc yêu cầu Copilot viết Unit Test tự động, sau đó chạy lệnh `unittest` để xác nhận mã nguồn đã hoạt động đúng.

D: > Downloads > lab51.py > ...

```
1  import inspect
2  import os
3  from dotenv import load_dotenv
4  from openai import OpenAI
5  from openai.types.chat import ChatCompletion
6
7  SURROUND = "You will be provided with a Python function enclosed with {{{ Function }}}."
8  SINGLE_TASK = "Your task is to generate Google Style docstring for it."
9
10
11 class Singleton(type):
12     _instances = {}
13
14     def __call__(cls, *args, **kwargs):
15         if cls not in cls._instances:
16             cls._instances[cls] = super(Singleton, cls).__call__(*args, **kwargs)
17         return cls._instances[cls]
18
19
20 def get_user_prompt(func: callable) -> str:
21     return f"""
22     FUNCTION: {{{{ inspect.getsource(func) }}}}}
23
24     GOOGLE STYLE DOCSTRING:
25     """
26
27
28 if __name__ == "__main__":
29     load_dotenv()
30     client: OpenAI = OpenAI(
31         api_key=os.getenv("OPENAI_API_KEY")
32     )
33
34     system_prompt = f"{SURROUND} {SINGLE_TASK}"
35     user_prompt = get_user_prompt(Singleton.__call__)
36
37     completion: ChatCompletion = client.chat.completions.create(
38         model="gpt-4o-mini",
39         messages=[
40             {"role": "system", "content": system_prompt},
41             {"role": "user", "content": user_prompt},
42         ],
43     )
44     print("Docstring:", completion.choices[0].message.content)
45
```

Hình 10: Mã nguồn Lab 5.1

```

1  √ from dataclasses import dataclass
2  from unittest import TestCase, main
3
4
5  √ class Singleton(type):
6      _instances = {}
7
8      √ def __call__(cls, *args, **kwargs):
9          √ if cls not in cls._instances:
10             cls._instances[cls.__name__] = super(Singleton, cls).__call__(*args, **kwargs)
11             return cls._instances[cls.__name__]
12
13
14  @dataclass
15  √ class Environment(metaclass=Singleton):
16      name: str = 'Production' # This is the default environment
17      version: int = 1
18
19
20  √ if __name__ == "__main__":
21      main()

```

Hình 11: Mã nguồn Lab 5.2

5.4. Kết luận

Kết thúc Chương 5, người đọc không chỉ nắm vững kỹ thuật viết code mà còn hiểu rõ cách "giao tiếp" hiệu quả với các mô hình ngôn ngữ lớn thông qua mã nguồn. Chương này khẳng định rằng sức mạnh của các công cụ GenAI phụ thuộc rất lớn vào khả năng cung cấp ngữ cảnh và tính cụ thể trong câu lệnh của người lập trình. Việc áp dụng thành thạo khung quy tắc 5 chữ S giúp giảm thiểu các kết quả sai lệch (hallucinations) và đảm bảo mã nguồn được sinh ra tuân thủ các tiêu chuẩn thiết kế chuyên nghiệp. Đây là bước đệm quan trọng để tiến tới Phần 2 của cuốn sách, nơi người dùng sẽ khám phá các kỹ thuật Prompt Engineering nâng cao và cách các mô hình như GPT-4 thực sự vận hành bên dưới lớp vỏ mã nguồn.

DANH MỤC TÀI LIỆU THAM KHẢO

[1] Packt Publishing, *Supercharged Coding with GenAI*. Birmingham, UK: Packt Publishing, 2025.

[2] Packt Publishing, "Supercharged Coding with Gen-AI Repository," 2025. [Online]. Available: <https://github.com/PacktPublishing/Supercharged-Coding-with-Gen-AI>.